

Hybrid Threat Detection Lab

Cloud attack emulation and detection engineering in AWS — Stratus Red Team, CloudTrail, Splunk, Sigma-to-SPL, and Python alert enrichment.

Tahmid Abtahee · github.com/tahosprojects · linkedin.com/in/tahmidabtahee · July 2026

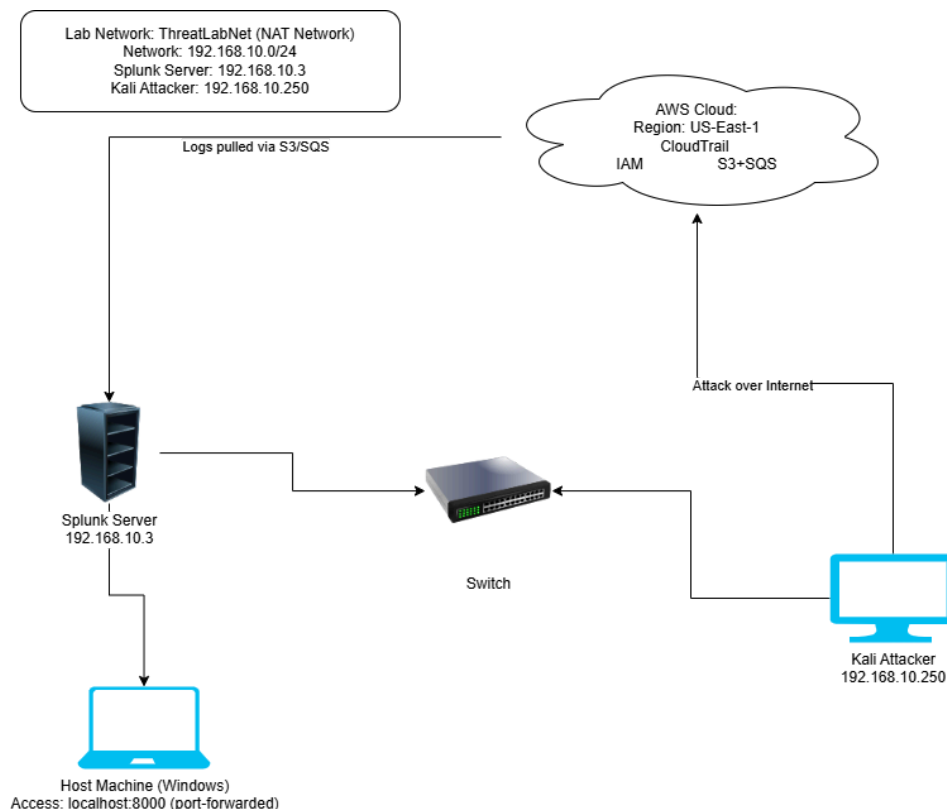
Overview

My earlier lab work centered on on-premises Windows telemetry — Active Directory, Sysmon, and endpoint forensics. This project extends that into the cloud and ties the two worlds together into one detection story. The objective was to treat AWS the way a real SOC does: stand up cloud infrastructure as code, generate genuine attacker activity against it, and then detect that activity by engineering my own detections against raw CloudTrail logs in Splunk — rather than leaning on a managed alerting service to do the thinking.

The result is an end-to-end pipeline. A Kali attacker detonates ATT&CK-mapped techniques against an AWS account, CloudTrail captures the API activity, the logs flow into a self-hosted Splunk instance, and a set of Sigma rules (converted to SPL) surface the malicious behavior. A Python script then enriches alerts with external threat intelligence to speed up triage.

Stack: AWS (CloudTrail, S3, SQS), Terraform, Splunk, Stratus Red Team, Sigma / sigma-cli, Python (boto3, requests), and Kali Linux — all on AWS free tier under a zero-dollar budget guardrail.

Architecture



CloudTrail records every API call in the account and delivers the logs to an S3 bucket. An SQS queue sits between S3 and Splunk: S3 publishes an event notification to the queue whenever a new log object lands, and Splunk's AWS add-on pulls from that queue. Decoupling ingestion this way means that if Splunk is slow or offline, messages wait in the queue instead of being lost, and a dead-letter queue catches anything that repeatedly fails to process. This mirrors how a production cloud SOC ingests activity at scale.

Cloud Infrastructure (Terraform)

I provisioned the AWS side entirely with Terraform rather than clicking through the console. Infrastructure as code makes the environment reproducible, reviewable, and trivial to tear down — which matters when you are working under a strict budget and want no billable resources left running by accident. A single terraform apply builds the whole cloud footprint; a single terraform destroy removes it.

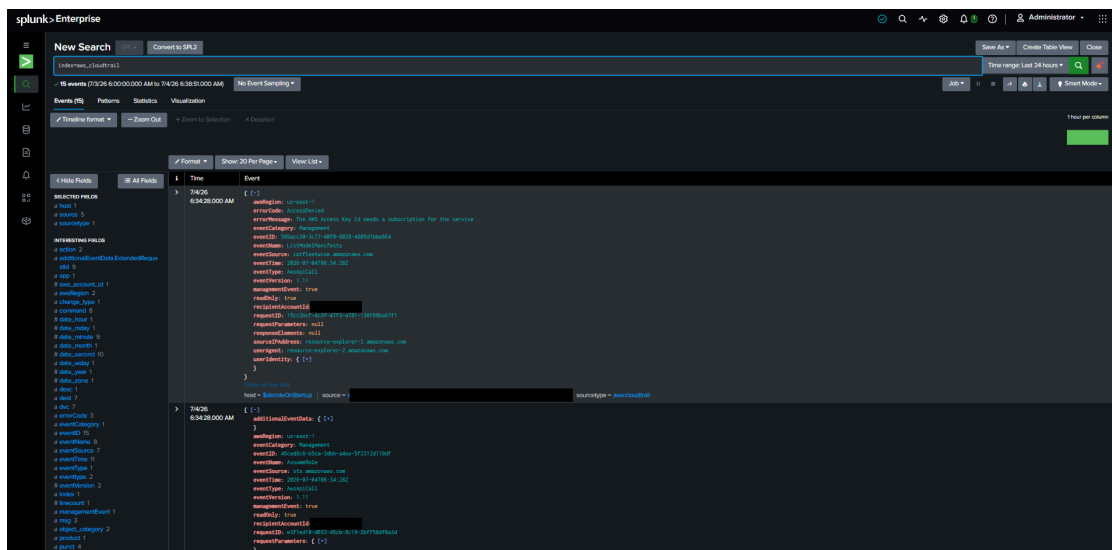
Terraform provisioned:

- An **S3 bucket** for CloudTrail log delivery, with a bucket policy granting the CloudTrail service permission to write to it.
- A **multi-region CloudTrail trail**, so activity in any region is captured — not just us-east-1.
- An **SQS queue** (300-second visibility timeout) subscribed to S3 object-created notifications, plus a **dead-letter queue** and redrive policy for messages that fail processing.

Splitting the configuration across separate files (provider, CloudTrail, and SQS) kept each concern isolated and easy to reason about.

Log Ingestion Pipeline

With the infrastructure live, I configured the Splunk AWS add-on to consume CloudTrail logs through the SQS-based S3 input built in Terraform. Splunk itself ran headless on an Ubuntu server, with the web interface reached from my host over port forwarding to keep the VM lightweight. After validating AWS connectivity from the CLI, CloudTrail events began landing in Splunk.



CloudTrail events indexed in Splunk, confirming the S3 → SQS → add-on pipeline is delivering logs.

A note on GuardDuty. I originally architected the lab to include Amazon GuardDuty, AWS's managed threat-detection service. Enabling it beyond the trial required upgrading to a paid tier, which conflicted with the zero-dollar constraint. Rather than treat that as a blocker, I dropped it deliberately: the whole point of this project is to demonstrate detection engineering, so running detections natively in Splunk against raw CloudTrail — instead of consuming pre-built GuardDuty findings — is the stronger showcase. Understanding the underlying detection logic is more valuable than wiring up a managed alert feed.

Adversary Emulation (Stratus Red Team)

To generate realistic attacker activity, I used Stratus Red Team, a purpose-built adversary-emulation tool that detonates granular, ATT&CK-mapped techniques against a live cloud account and cleans them up afterward. I ran five techniques spanning the attack lifecycle, from initial access through defense evasion.

Technique	ATT&CK	What it simulates
Console login without MFA	T1078.004	Sign-in with valid credentials but no MFA — the classic compromised-credential entry point.
EC2 enumeration from instance	T1580 / T1087	A rapid burst of discovery API calls mapping the environment.
Create IAM admin user	T1136.003	A new IAM user granted AdministratorAccess — a persistence backdoor.
Batch-retrieve secrets	T1555 / T1552	Bulk credential harvesting from AWS Secrets Manager.
Stop CloudTrail logging	T1562.008	Disabling logging to blind detection.

Each technique was detonated, confirmed in CloudTrail, detected in Splunk, and then cleaned up. The detections firing for each are shown below.

The screenshot shows the Splunk Search interface with a search query: `undertrack_cloudtrail | eventbase=Consolidate | "stratus-red-team-mfa"`. The search results show a single event from 7/8/26 at 12:10:13.000 AM. The event details are as follows:

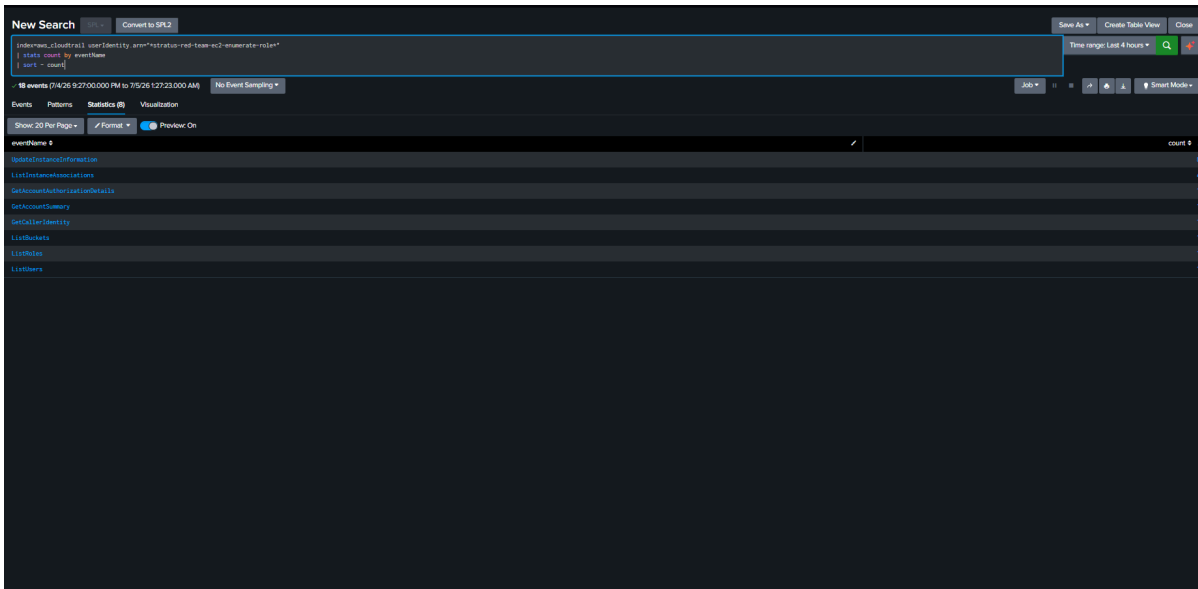
```

{
  "additionalEventData": {
    "authentication_method": "password",
    "eventCategory": "Management",
    "eventName": "ConsoleLogin",
    "eventSource": "signin.amazonaws.com",
    "eventType": "AWSConsoleLogin",
    "managementEvent": true,
    "readOnly": false,
    "recipientAccountId": "123456789012",
    "requestParameters": {
      "password": "123456789012"
    },
    "userIdentity": {
      "principalId": "AIDAJDPL4VEK5",
      "userName": "root",
      "type": "IAMUser",
      "userId": "AIDAJDPL4VEK5"
    }
  },
  "eventID": "123456789012",
  "eventName": "ConsoleLogin",
  "eventSource": "signin.amazonaws.com",
  "eventType": "AWSConsoleLogin",
  "managementEvent": true,
  "readOnly": false,
  "recipientAccountId": "123456789012",
  "requestParameters": {
    "password": "123456789012"
  },
  "userIdentity": {
    "principalId": "AIDAJDPL4VEK5",
    "userName": "root",
    "type": "IAMUser",
    "userId": "AIDAJDPL4VEK5"
  }
}

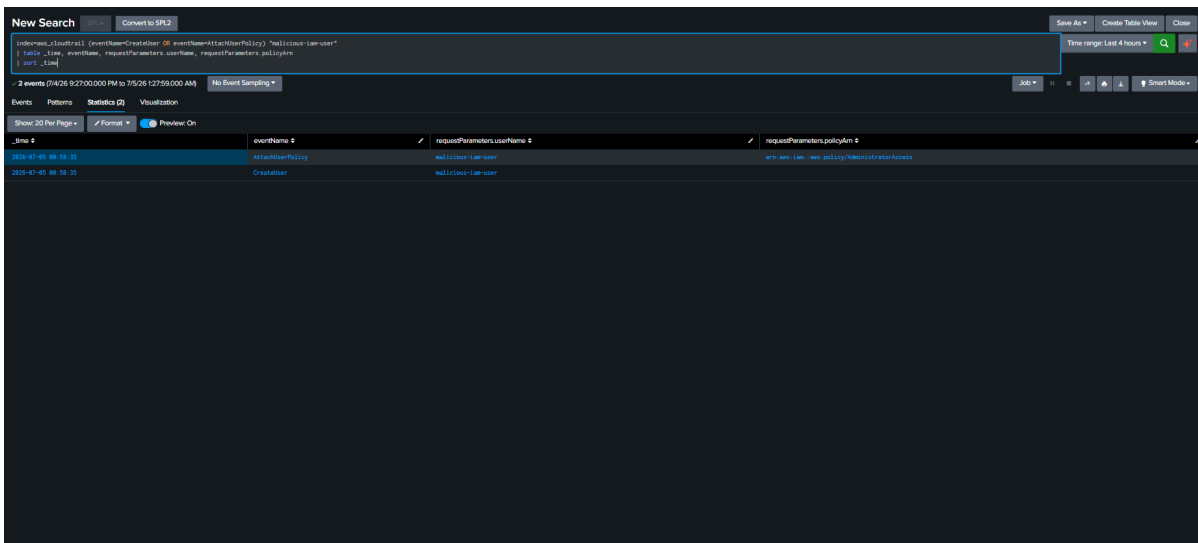
```

The event is categorized as 'Management' and 'AWSConsoleLogin'. The request parameters show a password login. The user identity is an IAM user named 'root'.

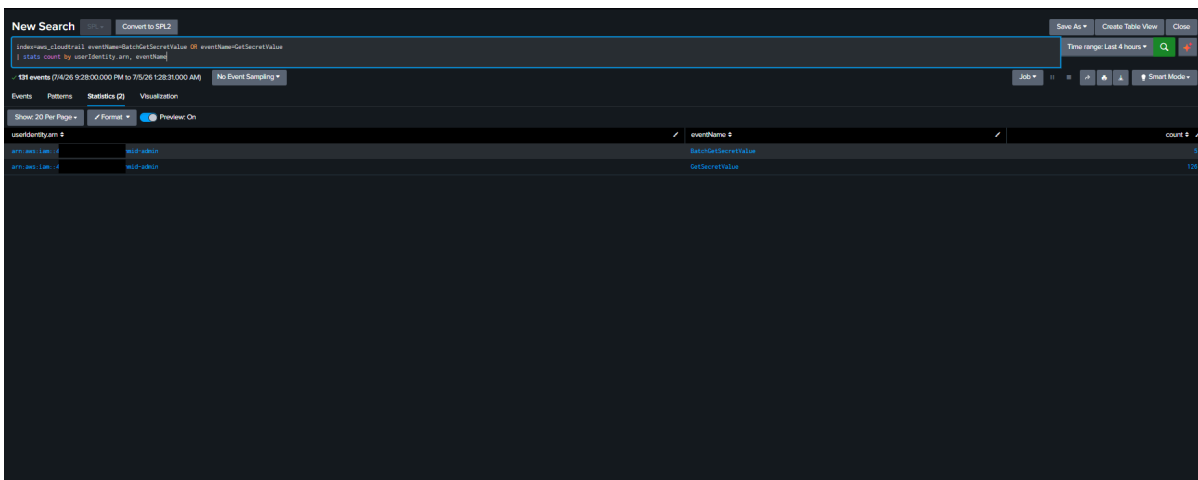
T1078.004 — a console login with MFAUsed = No surfaced in CloudTrail.



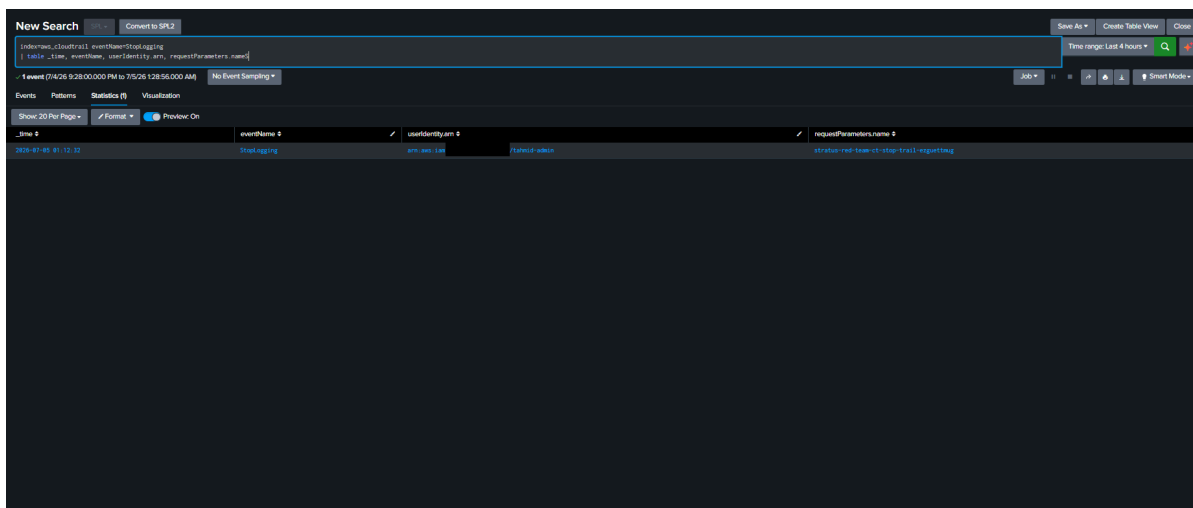
T1580 / T1087 — a burst of distinct discovery calls from a single identity.



T1136.003 — CreateUser immediately followed by an AdministratorAccess policy attachment.



T1555 / T1552 — repeated secret-retrieval calls from one identity.



T1562.008 — the StopLogging event captured before logging was disabled.

Resource note. My host has 16 GB of RAM, and running the Splunk and Kali VMs at 8 GB each starved it. I reallocated memory between the two to keep both responsive — a small but real constraint of running a multi-VM lab on a laptop.

Detection Engineering (Sigma → SPL)

Detecting the attacks was the core of the project. I wrote each detection as a Sigma rule — the vendor-agnostic detection format — and then converted it to Splunk SPL with sigma-cli and its Splunk backend. Writing in Sigma first keeps the logic portable to any SIEM and forces you to reason about the detection abstractly before binding it to one query language.

I deliberately built five rules that each use a different detection pattern, because real detection engineering is not one-size-fits-all:

Detection	Pattern	False-positive handling
Console login no-MFA	Single-event tripwire	Fires on any no-MFA console login; the only benign cause is an account genuinely lacking MFA — itself a finding to remediate.
Enumeration burst	Volume / variety aggregation	Counts distinct discovery calls per identity over 5 minutes; AWS service roles are filtered out and known inventory tools allowlisted.
Backdoor admin user	Correlated event sequence	Requires CreateUser and an AdministratorAccess attachment within 5 minutes; benign onboarding is separated by correlating with change tickets.
Bulk secrets retrieval	Rate / threshold per identity	Alerts when one identity pulls many secrets quickly; app and CI/CD service accounts that legitimately do so are allowlisted.
Stop logging	Single critical event	Any StopLogging / DeleteTrail / UpdateTrail fires immediately; planned maintenance is confirmed against change control.

Two problems separated a working detection from a naive one:

- **Service-role noise.** AWS's own service roles constantly issue List and Describe calls. Without filtering them, the enumeration-burst rule fires on routine AWS housekeeping. Excluding assumed-role ARNs that begin with [AWSServiceRoleFor](#) cleaned this up.
- **Legitimate-identity misuse.** The admin-creation and secrets techniques ran under my real administrative identity, not a separate attacker principal. That is realistic — a compromised admin looks like an admin — so these detections key on the *behavior* (a suspicious sequence or volume), not on assuming a “bad” identity. That is the harder and more valuable case.

```
(kali@kali)-[~]
$ pipx install sigma-cli
WARNING: Skipping setuptools as it is not installed.
Installed package sigma-cli 3.0.3, installed using Python 3.13.12
These apps are now available
- sigma
done! 🌟 🌟 🌟

(kali@kali)-[~]
$ sigma plugin install splunk
Successfully installed plugin 'splunk'
pySigma version is compatible with sigma-cli

(kali@kali)-[~]
$ sigma version
3.0.3 (online pypi.org: 3.0.3)

(kali@kali)-[~]
$ nano ~/detection1-nomfa.yml

(kali@kali)-[~]
$ sigma convert -t splunk ~/detection1-nomfa.yml
Usage: sigma convert [OPTIONS] INPUT...
Try 'sigma convert --help' for help.

Error:
Processing pipeline required by backend! Define a custom pipeline or choose a predefined one.

Get all available pipelines for splunk with:
sigma list pipelines splunk

If you never heard about processing pipelines you should get familiar with them
(https://sigmahq-pysigma.readthedocs.io/en/latest/Processing_Pipelines.html).
If you know what you're doing add --without-pipeline to your command line to suppress this error.

(kali@kali)-[~]
$ sigma list pipelines splunk
```

Identifier	Priority	Processing Pipeline	Backends	Plugin
splunk_windows	20	Splunk Windows log source conditions	splunk_sp12, splunk	splunk
splunk_sysmon_acceleration	25	Splunk Windows Sysmon search acceleration keywords	splunk_sp12, splunk	splunk
splunk_cim	20	Splunk CIM Data Model Mapping	splunk_sp12, splunk	splunk

```
(kali@kali)-[~]
$ sigma convert -t splunk --without-pipeline ~/detection1-nomfa.yml
Parsing Sigma rules [#####] 100%
eventSource="signin.amazonaws.com" eventName="ConsoleLogin" additionalEventData.MFAUsed="No"
```

Example Sigma-to-SPL conversion — the no-MFA console-login rule running as a native Splunk search. The full ruleset (Sigma and SPL) is in the repository.

Alert Enrichment Automation (Python)

Detecting an event is only half the job — an analyst still has to decide whether the source is actually hostile. I wrote a Python script (boto3 and requests) that takes IP addresses from CloudTrail activity and enriches them against AbuseIPDB, an open threat-intelligence source, scoring how likely each IP is malicious. The intent is to auto-generate an email alert whenever a flagged IP appears, so a reviewer gets context — reputation and report count — attached to the alert instead of looking it up by hand.

```
GNU nano 8.7.1 /home/splunky/enrich.py
import os
import requests
import time
import xml.etree.ElementTree as ET

requests.packages.urllib3.disable_warnings()

ABUSEIPDB_KEY = os.environ.get("ABUSEIPDB_KEY")

def enrich_ip(ip):
    url = "https://api.abuseipdb.com/api/v2/check"
    headers = {"key": ABUSEIPDB_KEY, "Accept": "application/json"}
    params = {"ipaddress": ip, "maxAgeInDays": 90}
    r = requests.get(url, headers=headers, params=params, verify=False)
    d = r.json()["data"]
    return {
        "ip": ip,
        "score": d["abuseConfidenceScore"],
        "reports": d["totalReports"],
        "country": d.get("countryCode", "N/A"),
        "isp": d.get("isp", "N/A")
    }

SPLUNK_HOST = "https://localhost:8089"
SPLUNK_USER = "admin"
SPLUNK_PASS =

SEARCH = 'search index=aws_cloudtrail event=ConsoleLogin | stats count by sourceIpAddress'

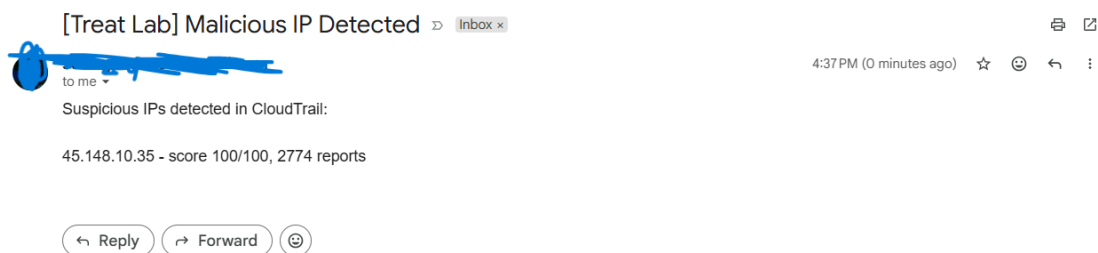
resp = requests.post(
    f"{SPLUNK_HOST}/services/search/jobs",
    auth=(SPLUNK_USER, SPLUNK_PASS),
    data={"search": SEARCH, "earliest_time": "-24h", "latest_time": "now"},
    verify=False
)

sid = ET.fromstring(resp.text).find("sid").text
print(f"Search job started: {sid}")

while True:
    status = requests.get(
        f"{SPLUNK_HOST}/services/search/jobs/{sid}",
        auth=(SPLUNK_USER, SPLUNK_PASS),
        params={"output_mode": "json"},

```

Enrichment script querying AbuseIPDB for IP reputation.



Enriched output identifying a flagged IP, ready to drive an email alert.

This is the piece that ties detection to response and shows how repetitive analyst triage can be automated.

Key Decisions & Lessons Learned

- **UTC vs. local time.** Stratus logs its actions in local time while CloudTrail records everything in UTC. When lining up “what I detonated” against “what the logs show,” that offset matters — correlating on the wrong clock makes real activity look like it is missing.
- **Detect behavior, not identity.** Because the most dangerous cloud attacks run under legitimate credentials, the strongest detections here trigger on patterns of activity rather than on who is acting.

- **Decouple ingestion with a queue.** Putting SQS and a dead-letter queue between S3 and Splunk means log delivery survives an outage instead of silently dropping events.
- **A missing tool can be a design choice.** Dropping GuardDuty for cost reasons became an opportunity to demonstrate the detection logic myself rather than hide it behind a managed service.

What This Demonstrates

This lab shows I can work across the full cloud detection lifecycle: provision AWS infrastructure as code, build a resilient log-ingestion pipeline, emulate ATT&CK techniques with production tooling, engineer portable detections with real false-positive tuning, and automate the enrichment that speeds up triage. It is built to reflect how a modern SOC actually operates — hybrid, log-driven, and detection-engineering-led.

This repository contains the full Terraform configuration, the Sigma ruleset, the SPL conversions, and the enrichment script.